

Отчет по лабораторной работе № 24 по алгоритмы и структуры данных

Студентка группы М8О-114БВ-25 Мозговая Жанна Евгеньевна, № по списку 13

Контакты www, e-mail, icq, skype mozgovay0607@gmail.com

Работа выполнена: «20» апреля 2026г.

Преподаватель: Сысоев Максим Алексеевич каф. 806

Входной контроль знаний с оценкой _____

Отчет сдан «21» апреля 2026 г., итоговая оценка _____

Подпись преподавателя _____

1. **Тема:** AST

2. **Цель работы:** Составить программу на С с выполнением арифметических выражений с помощью деревьев

3. **Задание (Вариант № 13):** Упрощение выражения.

4. **Оборудование:**

Оборудование ПЭВМ студента, если использовалось:

Процессор __12th Gen Intel(R) Core(TM) i5-1235U, 1300 МГц, ядер: 10, логических процессоров:12.

Монитор __BOE PnP Monitor _____

5. **Программное обеспечение (лабораторное):**

Программное обеспечение ЭВМ студента, если использовалось:

Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд bash версия 5.2.21.

Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4

Редактор текстов: VS code версия: 1.107.1.

Язык программирования: C

6. **Идея, метод, алгоритм** решения задачи (в формах: словесной, псевдокода, графической [блок-схема, диаграмма, рисунок, таблица] или формальные спецификации с пред- и постусловиями)

Идея: Заменить в арифметическом выражении все поддеревья умножения (*), в которых хотя бы один операнд — число 0 (пуль), на константу 0.

После этого всё выражение может частично или полностью упроститься.

Ключевые моменты:

Использование двоичного дерева для внутреннего представления выражения.

Рекурсивный обход дерева для обнаружения и замены умножений на 0.

Сохранение остальных операций без изменений.

Метод: синтаксический анализ + преобразование дерева

Алгоритм:

Вход: строка с арифметическим выражением (целые числа, операции +, -, *, /, скобки).

Выход: упрощённое выражение (все произведения с нулём заменены на 0).

Шаги:

Разбор строки

Используя алгоритм Дейкстры, разбить выражение на токены (числа, операторы, скобки).

Построить бинарное дерево с узлами-операциями и листьями-числами.

Упрощение дерева (рекурсивно)

Если узел — число → вернуть его.

Если узел — операция *:

Рекурсивно упростить левое и правое поддеревья.

Если левое или правое поддерево — 0 → вернуть узел-число 0.

Иначе → вернуть узел * с упрощёнными поддеревьями.

Если узел — другая операция (+, -, /):

Рекурсивно упростить оба поддерева.

Вернуть узел с упрощёнными поддеревьями.

Вывод результата

Выполнить инфиксный обход дерева (с учётом приоритетов для скобок).

Получить строку упрощённого выражения.

7. Сценарий выполнения работы [план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию].

План работы:

- ✓ Анализ задачи
- ✓ Проектирование решения
- ✓ Написание чернового варианта
- ✓ Тестирование
- ✓ Отладка и оптимизация
- ✓ Приведение к стилю СІ
- ✓ Готовый код

Первоначальный текст программы в черновике:

// Черновик. Программа упрощения выражений: умножение на 0 -> 0

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
```

```
// Узел дерева
struct Node {
    int isNum;      // 1 - число, 0 - операция
    int value;      // если число
    char op;        // если операция
    struct Node *left, *right;
};
```

```
// Стек
struct Stack {
    struct Node** arr;
    int top;
    int cap;
};
```

```
// Создать узел-число
struct Node* makeNum(int v) {
    struct Node* n = malloc(sizeof(struct Node));
    n->isNum = 1;
    n->value = v;
    n->left = n->right = NULL;
    return n;
}
```

```
// Создать узел-операцию
```

```

struct Node* makeOp(char o, struct Node* L, struct Node* R) {
    struct Node* n = malloc(sizeof(struct Node));
    n->isNum = 0;
    n->op = o;
    n->left = L;
    n->right = R;
    return n;
}

// Упрощение
struct Node* simplify(struct Node* root) {
    if (!root) return NULL;
    if (root->isNum) return root;

    root->left = simplify(root->left);
    root->right = simplify(root->right);

    if (root->op == '*') {
        // Если слева 0 или справа 0
        if (root->left && root->left->isNum && root->left->value == 0)
            return makeNum(0);
        if (root->right && root->right->isNum && root->right->value == 0)
            return makeNum(0);
    }
    return root;
}

```

// Пока пропущены: parseExpression, toInfix, main (набросок)
 // План main:
 // 1. Ввод строки
 // 2. Построить дерево (parse)
 // 3. Упростить
 // 4. Вывести результат

Тесты:

План тестирования:

Модульное тестирование (пофункционально)

makeNum, makeOp – проверка создания узлов.

simplify на маленьких деревьях: 0*a, a*0, a*b.

toInfix – обратное преобразование.

Интеграционное тестирование

Запуск на всех примерах из таблицы.

Сравнение вывода с ожидаемым.

Граничные случаи

Пустая строка (недопустимо, но можно обработать).

Очень длинное выражение.

Отрицательные числа (если поддерживаются).

Проверка памяти

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_EXPR 256

typedef enum { NUM, OP } NodeType;

typedef struct Node {
    NodeType type;
    char op;           // для OP: +, -, *, /
    int value;         // для NUM (целый тип)
    struct Node *left, *right;
} Node;

// Стек для построения дерева
typedef struct {
    Node** data;
    int top;
    int capacity;
} Stack;

Stack* createStack(int cap) {
    Stack* s = (Stack*)malloc(sizeof(Stack));
    s->data = (Node**)malloc(sizeof(Node*) * cap);
    s->top = -1;
    s->capacity = cap;
    return s;
}

void push(Stack* s, Node* n) {
    if (s->top + 1 < s->capacity)
        s->data[++s->top] = n;
}

Node* pop(Stack* s) {
    if (s->top >= 0)
        return s->data[s->top--];
    return NULL;
}

int isEmpty(Stack* s) {
    return s->top == -1;
}

Node* createNumNode(int val) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->type = NUM;
    n->value = val;
    n->left = n->right = NULL;
    return n;
}

Node* createOpNode(char op, Node* left, Node* right) {
    Node* n = (Node*)malloc(sizeof(Node));
    n->type = OP;
    n->op = op;
    n->left = left;
    n->right = right;
    return n;
}

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

void applyOp(Stack* values, char op) {
    Node* right = pop(values);
    Node* left = pop(values);
```

```

    push(values, createOpNode(op, left, right));
}

// Построение дерева из выражения (алгоритм Дейкстры)
Node* parseExpression(const char* expr) {
    Stack* values = createStack(MAX_EXPR);
    Stack* ops = createStack(MAX_EXPR);

    for (int i = 0; expr[i]; i++) {
        if (isspace(expr[i])) continue;

        if (isdigit(expr[i])) {
            int num = 0;
            while (isdigit(expr[i])) {
                num = num * 10 + (expr[i] - '0');
                i++;
            }
            push(values, createNumNode(num));
            i--;
        }
        else if (expr[i] == '(') {
            push(ops, createOpNode('(', NULL, NULL));
        }
        else if (expr[i] == ')') {
            while (!isEmpty(ops) && ops->data[ops->top]->op != '(') {
                Node* opNode = pop(ops);
                applyOp(values, opNode->op);
                free(opNode);
            }
            if (!isEmpty(ops)) free(pop(ops));
        }
        else if (strchr("+-*/", expr[i])) {
            while (!isEmpty(ops) && ops->data[ops->top]->op != '(' &&
                precedence(ops->data[ops->top]->op) >= precedence(expr[i])) {
                Node* opNode = pop(ops);
                applyOp(values, opNode->op);
                free(opNode);
            }
            push(ops, createOpNode(expr[i], NULL, NULL));
        }
    }

    while (!isEmpty(ops)) {
        Node* opNode = pop(ops);
        applyOp(values, opNode->op);
        free(opNode);
    }

    Node* root = pop(values);
    free(values->data); free(values);
    free(ops->data); free(ops);
    return root;
}

Node* simplify(Node* node) {
    if (!node) return NULL;

    node->left = simplify(node->left);
    node->right = simplify(node->right);

    if (node->type == OP && node->op == '*') {
        // Проверяем, есть ли 0 в левом или правом поддереве
        if ((node->left && node->left->type == NUM && node->left->value == 0) ||
            (node->right && node->right->type == NUM && node->right->value == 0)) {
            freeTree(node->left);
            freeTree(node->right);
            node->type = NUM;
            node->value = 0;
            node->left = node->right = NULL;
            return node;
        }
    }
    return node;
}

```

```

void printTree(Node* node, int level) {
    if (!node) return;
    for (int i = 0; i < level; i++) printf(" ");
    if (node->type == NUM) {
        printf("%d\n", node->value);
    } else {
        printf("%c\n", node->op);
        printTree(node->left, level + 1);
        printTree(node->right, level + 1);
    }
}

void toInfix(Node* node, char* buf, int* pos) {
    if (!node) return;
    if (node->type == NUM) {
        *pos += sprintf(buf + *pos, "%d", node->value);
    } else {
        if (node->left) {
            if (node->left->type == OP && precedence(node->left->op) < precedence(node->op))
                *pos += sprintf(buf + *pos, "(");
            toInfix(node->left, buf, pos);
            if (node->left->type == OP && precedence(node->left->op) < precedence(node->op))
                *pos += sprintf(buf + *pos, ")");
        }
        *pos += sprintf(buf + *pos, " %c ", node->op);
        if (node->right) {
            if (node->right->type == OP && precedence(node->right->op) < precedence(node->op))
                *pos += sprintf(buf + *pos, "(");
            toInfix(node->right, buf, pos);
            if (node->right->type == OP && precedence(node->right->op) < precedence(node->op))
                *pos += sprintf(buf + *pos, ")");
        }
    }
}

void freeTree(Node* node) {
    if (!node) return;
    freeTree(node->left);
    freeTree(node->right);
    free(node);
}

int main() {
    char expr[MAX_EXPR];
    printf("Программа упрощения арифметических выражений\n");
    printf("Введите арифметическое выражение (с *, +, -, / и целыми числами):\n");
    printf("Примеры: 5 * 0 + 3, (2 + 3) * 0, 0 * (4 + 5)\n");
    printf("-> ");
    fgets(expr, MAX_EXPR, stdin);
    expr[strcspn(expr, "\n")] = '\0';

    printf("\nИсходное выражение: %s\n", expr);

    Node* root = parseExpression(expr);
    printf("\nДерево исходного выражения:\n");
    printTree(root, 0);

    Node* simplified = simplify(root);
    printf("\nУпрощённое дерево (убраны произведения с 0):\n");
    printTree(simplified, 0);

    char outBuf[MAX_EXPR];
    int pos = 0;
    toInfix(simplified, outBuf, &pos);
    printf("\nУпрощённое выражение: %s\n", outBuf);

    freeTree(simplified);
    printf("Программа завершена.\n");

    return 0;
}

```

9. Дневник отладки должен содержать дату и время сеансов отладки, и основные события(ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. Замечания автора по существу работы: замечания отсутствуют

11. Выводы: В ходе выполнения лабораторной работы был успешно разработан и реализован алгоритм для обмена элементов главной и побочной диагоналей квадратной матрицы.. Основным достижением работы стала реализация эффективного алгоритма обмена диагоналей, который выполняется за линейное время $O(n)$ относительно размерности матрицы. Алгоритм корректно обрабатывает как четные, так и нечетные размеры матриц, учитывая особенность центрального элемента при нечетной размерности, который не требует обмена. В процессе разработки были освоены практические навыки работы с файловым вводом-выводом в языке C, динамическим выделением памяти для двумерных массивов и организацией безопасного доступа к данным.

Подпись студента _____

